TWIN-64
Architecture Document

Helmut Fieres
August 22, 2025

Contents

Introduction

Designers of CPUs in the seventies and early eighties almost all used a microcoded approach with hundreds of instructions. RISC was just beginning to emerge. Registers were typically not fully general-purpose but often had special functions or meanings, and many instructions were rather complicated, though designers believed them useful. Compiler writers, however, often used only a small subset, ignoring these complex instructions because they were too specialized. In the late eighties and nineties the shift moved to RISC-based designs, with large sets of registers, fixed instruction word lengths, large virtual memory address ranges, and instructions that were in general much more pipeline-friendly. Today, processors are highly complex, with superscalar deep pipelines and multilevel cache hierarchiesjust a few of the hallmarks of modern CPU design. CPU designs have become so large and complex that no single person can completely understand them. But what if there were a simple design that one person could fully understand?

Welcome to Twin-64. Twin-64 is a simple 64-bit CPU with a registermemory model and segmented virtual memory. The design is heavily influenced by Hewlett-Packards PA_RISC architecture, which was initially a 32-bit RISC-style load/store machine. Almost all of the key architectural features come from there. However, other processorssuch as the Data General MV8000, the DEC Alpha, and the IBM PowerPCwere also influential or at least investigated for their design choices. In addition, the Blitz-64 architecture was studied. Blitz-64, a design developed at Portland State University, offered an interesting approach in that it only supported 64-bit signed arithmetic, avoiding common errors from mixing signed and unsigned arithmetic.

While it is not a design goal to build an industry-strength, competitive CPU, the CPU should nevertheless be useful and largely follow established design practices. For example, instructions should not be invented just because they seem useful, but rather designed with the assembler and compilers in mind. Instruction set design is CPU design. Registermemory architectures were typically microcoded, complex-instruction machines. In contrast, Twin-64 instructions will be hardwired and are in general pipeline-friendly, avoiding data and control hazards and stalls where possible.

Where does the name Twin-64 come from? Obviously, the 64 indicates that the CPU is a 64-bit CPU. The Twin part will become clearer when implementations of the CPU are described in later chapters. One implementation represents a dual-issue instruction design with two ALUs for computation. Nevertheless, a very basic implementation could just offer single-issue, single-ALU execution as well.

This document describes the overall architecture, instruction set, and runtime model for Twin-64. It is structured into several parts. The first part gives an overview of the system organization and architecture. The main part of the document presents the instruction set. Memory reference instructions, immediate instructions, branch instructions, computational instructions, and system control instructions are described in detail. These chapters are the authoritative reference for the instruction set. The next part presents the major components such as the processor itself, the I/O module architecture, and finally the runtime model and calling conventions. An appendix provides additional design details and complementary notes.

System Organization

This chapter introduces Twin-64 by highlighting the major components and design decisions taken.

A RegisterMemory Architecture

Modern RISC CPUs are typically load/store designs and feature a large number of general-purpose registers. Operations take place between registers. Twin-64 follows a slightly different route. It has fewer general registers, and in addition to registerregister computation, a register-memory model is also supported. However, there are no instructions that read from and write to memory in a single instruction cycle, since this would complicate the design considerably.

Twin-64 implements a **register-memory architecture** offering only a few addressing modes for the operand, combined with a fixed instruction word length. For most instructions one operand is a register, while the other is a flexible operand that can be an immediate, another register, or a memory address from which data is obtained or stored. The result is placed in the register, which is also the first operand. These types of machines are also called two-address machines.

In contrast to similar historical registermemory designs, Twin-64 has no operations that both read and write memory in the same instruction. For example, there is no increment memory instruction, since this would require two memory operations and is generally not pipeline-friendly. Memory data access is performed at the word, half-word, or byte level. Besides the implicit operand fetch in a computational instruction, there are dedicated load/store instructions. In addition to the registerimmediate and registermemory operand modes, one operand mode supports the three-operand model ($R_s = R_a \,\mathrm{OP}\, R_b$), specifying two source registers and a result register. This allows Twin-64 to support both registermemory and load/store operation styles.

Processor

The Twin-64 processor is only one element of a complete system. A system also includes memory, I/O modules, adapters, and interconnecting buses. The Central Processing Unit (CPU) includes a general register set and machine state registers. To support virtual memory addressing, a hardware translation lookaside buffer (TLB) is included to provide virtual-to-physical address translation. Instruction and data caches are optional, but for performance reasons are also key components of the processor. A processor will always emit virtual addresses that must be translated into physical addresses by the TLB. A Twin-64 system can consist of multiple processors.

Virtual Memory

Twin-64 offers a large global address range, organized in segments. A segment number and offset together form the global virtual address. Segments are also associated with access rights and protection identifiers. The CPU itself can run in user or privileged mode. Twin-64 always generates virtual addresses at the CPU level. The global virtual memory range is a 52-bit space organized into a 20-bit segment ID and a 32-bit byte offset. There are 2^{20} segments with 2^{32} bytes each.

Virtual addresses are translated to physical addresses by the TLB when memory is referenced. For memory management purposes, the virtual address range can also be viewed as a set of pages. A segment therefore contains 2^{18} pages of 16 KB each. A virtual page number, combining the segment and the page number within that segment, is a 2^{38} value. Address translations are made at the page level.

The first segments in the virtual address space are special in that they also represent the physical address range. A virtual address in that range bypasses the TLB and accesses physical memory directly.

A TLB can optionally support different page sizes in addition to the architected 16 KB page size. Supported sizes are 16 bytes, 256 KB, 4 MB, and 64 MB. It is very common for the operating system and data to be mapped in consecutive physical pages, which can then be covered by a larger page size.

Security and Protection Support

Twin-64 implements a protection model along two dimensions. The first dimension is **access control** and privilege-level checking. Each page is associated with a page type. Page types include read-only, readwrite, execute, and gateway. There are two privilege levels: user and supervisor mode. Access type and privilege level together form the access control information, which is checked for each instruction and data memory access.

The second dimension of protection is the **protection ID**, which is the segment ID itself. Twin-64 allows a set of segment IDs to be recorded in processor control registers. For each access to a segment that has the protection check bit set, one of the protection ID control registers must match the segment ID. If not, a protection violation trap is raised.

Privilege Changes

Instructions can only access data or execute instructions when the privilege level matches the required privilege level. Two or four levels are the most common implementations. Changing between levels is often done with a kind of trap operation to higher-privilege software, for example the operating system kernel. However, traps are expensive, as they cause the CPU pipeline to be flushed when entering a trap handler.

Twin-64 implements gateways for privilege promotion. A special option on the unconditional branch instruction raises the privilege level when the instruction is executed on a gateway page,

setting the privilege level to that of the gateway page. Returning from a higher privilege level to a code page with lower privilege level automatically resets the lower privilege level. A branch to a higher-privilege page that is not a gateway page results in a privilege violation trap.

Debugging Support

A fundamental requirement is the ability to set instruction and data breakpoints. An instruction breakpoint is triggered by encountering the `TRAP` instruction. The break instruction causes an instruction break trap and passes control to the trap handler. Since break instructions are normally not present in the instruction stream, they must be inserted dynamically in place of the instruction normally found at that address. Upon continuation, if the breakpoint is still valid, it must be reinserted after the original instruction is executed. To facilitate this mechanism, a bit in the status word provides an easy way to trap again after the instruction has been executed.

Data breakpoints are traps taken when a data reference to a page occurs. They do not modify the instruction stream. Depending on the data access, the trap may occur before or after the instruction that accesses the data. A write operation is trapped before the data is written, while a read instruction is trapped after the original instruction has executed.

Input/Output

Twin-64 features a memory-mapped I/O architecture. A portion of the physical address space is dedicated to the I/O subsystem. Within this address range, I/O modules respond to memory load and store instructions directed to their assigned addresses. When implementing a driver, the standard page-level protection mechanism is applied during address translation. A driver, typically implemented to run in privileged mode, can also grant a user program direct control by mapping a user-level I/O page into virtual space without compromising system integrity.

Interrupt System

Interrupts and exceptions are events that occur asynchronously to instruction execution. Depending on the type, they occur either during the execution of an instruction or between instructions. An example of the former is an arithmetic overflow trap; an example of the latter is an external interrupt. Depending on the interrupt or exception type, the instruction is either restarted or execution continues with the next instruction.

Twin-64 implements a simple interrupt system. Each processor features an interrupt input register and an interrupt mask register. When an I/O module receives a command, it also receives information about which interrupt bit to set and which target module ID to send the interrupt to upon completion of the request. The interrupt data is compared to the interrupt mask register. If the particular bit is enabled and interrupts are globally enabled, the target processor enters the interrupt handler.

Since raising an interrupt is simply a matter of storing the interrupt request into the processors interrupt register, I/O modules as well as processors themselves can raise interrupts on a target

processor. Processors can also send interrupts to their peers, enabling low-level processor-to-processor communication.

Instruction Set

Twin-64 features a simple and efficient instruction set. All instructions are of fixed length, specifically a 32-bit word. The total number of instructions is relatively small; however, many instructions encode execution options that make them highly versatile. Great care is taken to ensure that these options do not significantly complicate the pipeline or increase the overall data path length. Instead of supporting a wide variety of addressing modes, as commonly found in CISC-style CPUs, Twin-64 provides two virtual memory addressing modes based on a simple base-offset calculation. No indirection or addressing mode that requires reading a memory operand to compute an address is part of the architecture.

The instruction set is divided into five groups.

- **Memory reference instructions:** Load and store operations for virtual and physical memory.
- **Immediate instructions:** Building immediate values up to 64 bits.
- **Control flow instructions:** Conditional and unconditional branches.
- **Computational instructions:** Arithmetic, Boolean, and bit operations such as extract and deposit.
- **System control instructions:** Managing hardware resources such as caches and TLBs, register movement, traps, and interrupt handling.

Computational Instructions

Arithmetic, logical, and bit-field instructions form the computational class. Twin-64 supports only 64-bit signed arithmetic operations. Any numeric overflow results in a trap.

Binary operations allow negation of either input operand as well as the output. Operand negation provides an efficient mechanism for building complementary bit masks. Bit-field instructions perform extraction and deposit operations, as well as double-register shifts. These serve as the basis for all shift and rotate operations.

Immediate Instructions

Several instructions include bit fields for representing immediate values within the instruction word. Fixed-length instruction word architectures, however, do not allow embedding a full 64-bit immediate in a single instruction. Instead, Twin-64 uses a sequence of two or three instructions to assemble a complete value from parts.

Immediate instructions also support 32-bit address arithmetic. Address calculations wrap around within the 32-bit offset field and never overflow into the segment identifier portion.

Memory Reference Instructions

Memory reference instructions transfer data between memory and general registers. Supported transfer sizes are word, half-word, and byte. In this sense, the architecture resembles a typical load/store design. However, unlike pure load/store architectures, Twin-64 allows many instructions with an operand field to directly access memory. No instruction, however, both reads and writes data memory in the same operation.

With the exception of load-reserved and store-conditional, memory reference instructions support base register modification. When enabled, the base register used for address computation is updated before or after the memory access. A negative offset is applied before address computation, while a positive offset is applied afterward. Indexed addressing further extends the reach by scaling the index according to the data type.

Control Flow Instructions

Control flow instructions are divided into unconditional and conditional branches. Unconditional branches may optionally save the return point in a general register. Returning from a subroutine is accomplished by branching to the saved register value.

Conditional branch instructions compare two operands and transfer control if the condition is satisfied. This eliminates the need for a separate compare instruction followed by a branch.

Branches are further divided into instruction-address-relative and global-address-relative types. The first computes the target relative to the current instruction address, while the second uses a global virtual address as the base.

System Control Instructions

System control instructions manage processor resources such as the TLB, cache, and control registers. Instructions exist for direct access to control registers. The TLB can be managed through insert and remove operations. Cache management includes instructions to flush and purge data. Additional instructions generate traps or provide diagnostic functions for controlling and examining processor state. Most instructions in this group require execution in privileged mode.

Processing Resources

This chapter describes the processing resources and data types in a Twin-64 system.

General Registers

Twin-64 features a set of 16 general purposes registers. They are used for all computations.

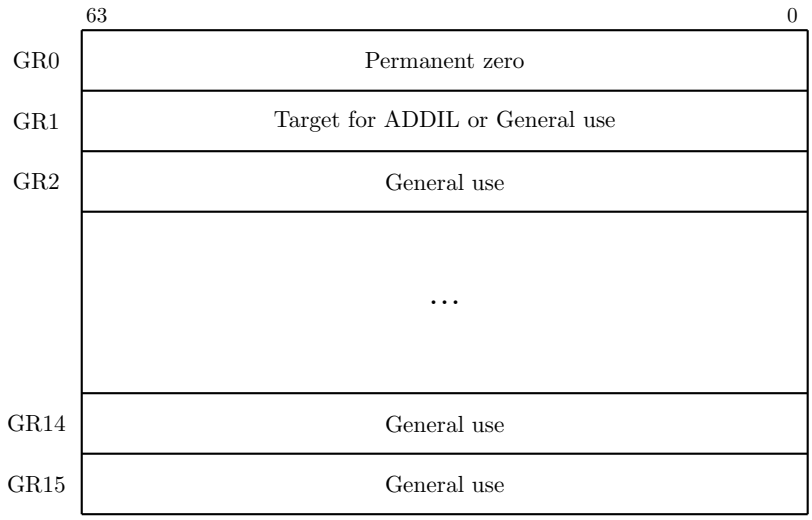


Figure 1: General registers

Some general registers have dedicated purposes. Register zero always returns a zero value when read, and writing to it has no effect. Although the CPU has only 16 general registers, implementing such a register greatly simplifies instruction design, for example by providing a target when the result is not needed. General register one is a scratch register and also serves as an implicit target for certain instructions. Other general registers may have dedicated roles as defined by the runtime architecture conventions.

Control Registers

Twin-64 has 16 defined control registers, numbered `CR0` to `CR15`. Two instructions allow moving data between a control register and a general register in either direction. A move to a control register always transfers the full 64-bit word. A move from a control register to a general register transfers the value right-aligned to the size of the control register. The following table shows the control registers defined for Twin-64.

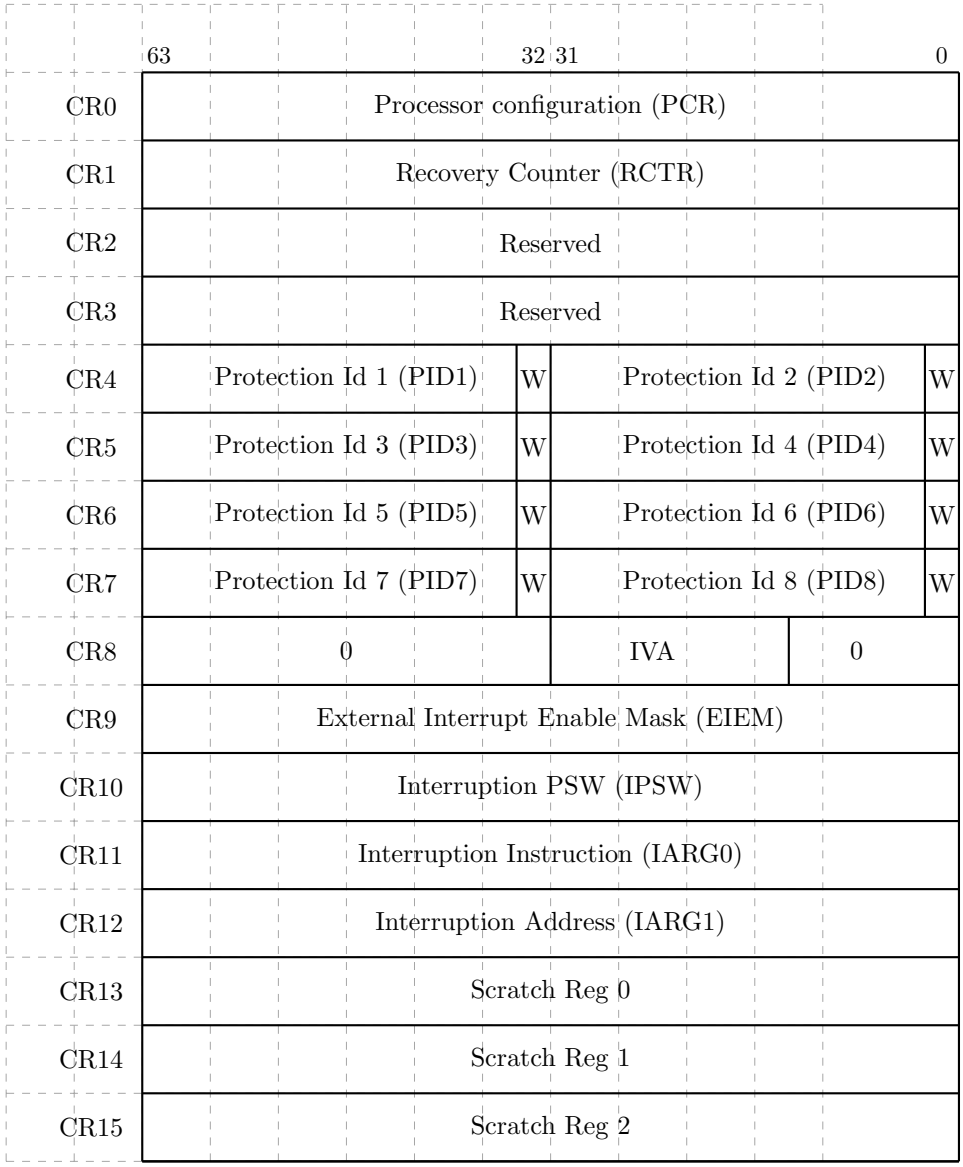


Figure 2: Control registers

Processor Configuration Register

The processor configuration register contains information about the current system and processor hardware setup. Some of its fields are read-only and set directly by the hardware, while others can be modified by processor-dependent code routines.

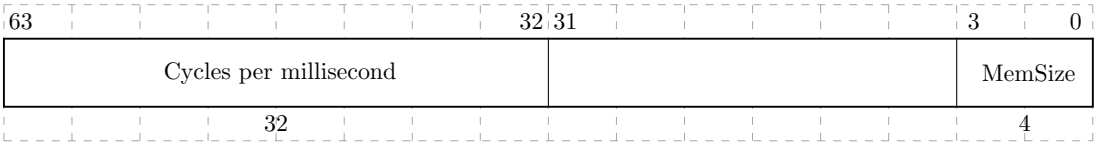


Figure 3: Processor Configuration Register

Table 1: Processor Configuration Register Fields

Field	Purpose
Memory Size	The available physical memory in 4Gbyte increments. The minimum size is 4Gbyte, encoded as a zero. The maximum size is 64Gbyte encoded as a 15.
Flags	e.g default endian, etc.
Architecture Id	...
Processor Id Id	...
Processor Core Num	...
Cycles per millisecond	...
...	...

Recovery Counter

decrements when the PSW status bit is set.

Protection Id Registers

Twin-64 allows to record a set of up to 8 protection Id values in the processor control registers `CR4` to `CR7` which are accessible to the currently executing process. The protection Id is identical to a segment Id. For each access to a segment that has the protection check bit set, one of the protection Id control registers must match the segment Id. If not, a protection violation trap is raised. The rightmost bit of each of the eight PIDs is the write disable (W) bit. When the bit is set, that PID cannot be used to grant write access. See also the chapter on addressing and access control.

Interrupt Vector Address

The Interruption Vector Address (IVA) control register, `CR8`, holds the base address of an array of service routines assigned to the different interruption classes. The lower 14 bits of the IVA are reserved and must be set to zero, meaning any address written to it must be aligned to a page boundary. The IVA is always located in the first segment, segment 0. The array of interruption service routines is indexed by interruption numbers, as described in the chapter on interrupts.

External Interrupt Enable Mask

The External Interrupt Enable Mask (EIEM) is stored in `CR15)`. It is a 64-bit register, with each bit corresponding to one of the 64 external interrupts. A cleared bit in the EIEM masks the external interrupt associated with that position.

Interruption PSW

Upon an interrupt, the address of the interrupted instruction and the processor status bits are saved to this control register.

Interruption Argument Registers

The Interruption Argument Registers (`CR11` and `CR12`) are used to pass an instruction and a virtual address to an interruption handler. The values of these registers for each interruption class are described in the chapter on interrupts.

Scratch Registers

Control registers `CR13`, `CR14`, and C`CR15` are scratch registers used by interrupt handlers or similar routines. `CR13` and `CR14` are privileged and can only be accessed from code running at a privileged level, whereas `CR15` is accessible from any privilege level.

Program State Register

The processor state register contains the virtual address of the current instruction along with the processor status flags.

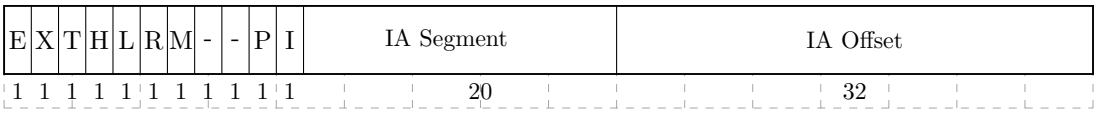


Figure 4: Program State Register

Instruction Address

Processor Status

Table 2: Status Field Bits

Bit	Purpose
M	High priority machine check. When set, high-priority machine checks are masked. This bit is set after an HPMC and cleared after all other interruptions.
E	Little endian access enable. When set, memory access is in little endian format, else big endian format. Endian support is an optional feature. If not implemented, all access is in big endian format.

Continued on next page

Bit	Purpose
R	Recovery counter enable. When set...
X	When set, the processor runs in privileged mode.
T	When set, taken branch traps are enabled. Any branch taken will result in a trap.
H	When set, a higher-privilege transfer trap occurs whenever the following instruction is of a higher privilege.
L	When set, a lower-privilege transfer trap occurs whenever the following instruction is of a lower privilege.
I	When set, external interrupts are enabled.
V	Reserved.
P	Protection identifier validation enable. When set instruction and data references are checked for valid protection identifiers (PIDs). The `PRB` instruction checks for valid PIDs without causing a trap.
D	Data memory break disable. The D-bit is cleared after the execution of each instruction, except for the `RFI` instruction which may set it. When set, data memory break traps are disabled. This bit allows a simple mechanism to trap on a data store and then proceed past the trapping instruction.
...	...

Data Types

The fundamental data type is a 64-bit signed integer, with the most significant bit on the left and the least significant bit on the right. All computations are performed on 64-bit signed integers. Memory accesses can be performed at the granularity of a doubleword, word, halfword, or byte. Data loaded as a word, halfword, or byte is sign-extended to 64 bits. Store operations write the corresponding value to memory without modification.

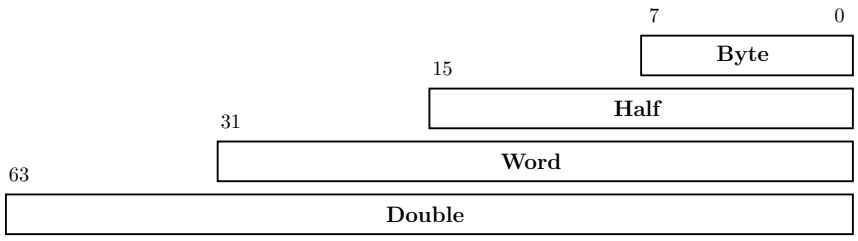


Figure 5: Data types

All memory addresses are expressed as bytes addresses. Address computation uses a 32-bit unsigned math, i.e. numbers wrap around in a 32-bit space and never cross segment boundaries.

Byte Ordering

Twin-64 is a big-endian machine. The following figure shows the memory access for load operations.

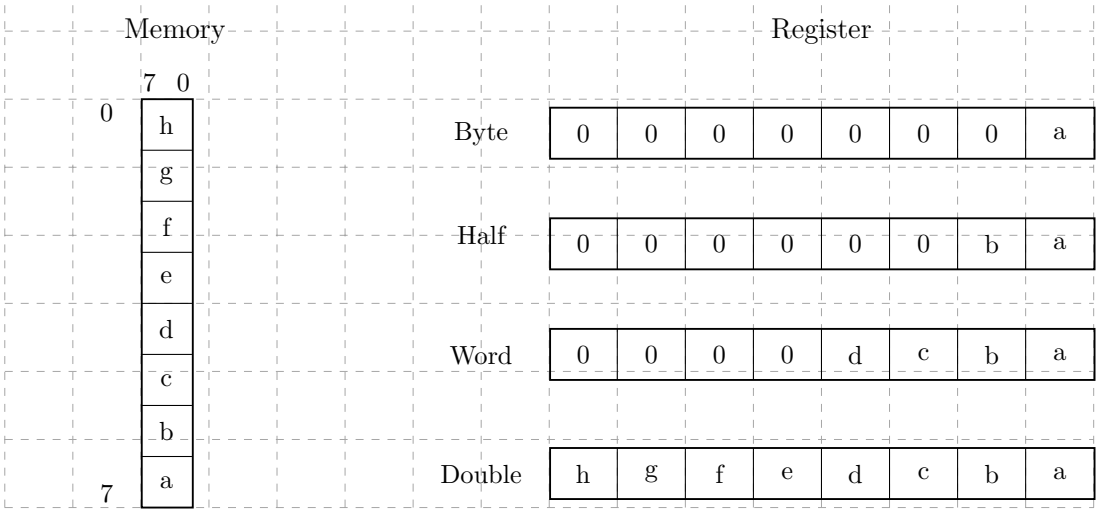


Figure 6: Big endian mode

The processor status register reserves a bit position, "E", for future support of mixed big and little endian memory access. Currently this bit is always set to zero.

Translation Lookaside Buffer

Virtual address translations are stored in a translation lookaside buffer (TLB). The TLB is essentially a cache of page table entries representing the virtual pages currently mapped to physical addresses. Depending on the hardware implementation, there may be a single combined TLB or separate instruction and data TLBs. Each TLB entry stores the virtual page number (VPN) along with its attributes and the corresponding physical page number (PPN). The current architecture supports a 36-bit physical address space, allowing a maximum of 64GB of memory.

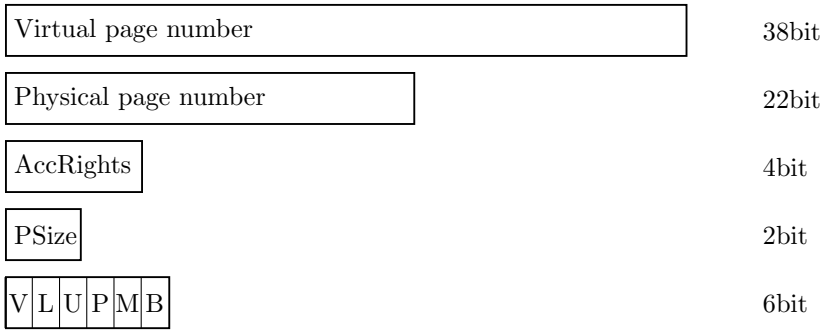


Figure 7: Tlb info

To reduce TLB processing overhead, the TLB is capable of storing virtual address ranges. There is support for a large page size of 16Kb, 256 Kb, 4Mb and 64 Mb.

Table 3: TLB Fields

Field	Purpose
VPN	The virtual page number.
PPN	The physical page number.
Access Rights	Encodes the access type (read, write, and execute) and the required privilege level for the operation.
Page Size	2-bit field encoding the supported page size: 0 = 16 KByte, 1 = 256 KByte, 2 = 4 MByte, 3 = 64 MByte.
V	Indicates if the entry is valid.
L	When set, the TLB entry is locked and will not be selected during replacement.
U	When set, the page represented by the TLB entry is not cached.
P	When set, the page has protection checking enabled.
M	When set, the page has been modified.
B	When set, a branch instruction executed from this page will cause a branch-taken trap.

The first segments, segment 0 to 15, are special in that they bypass the TLB entirely. A virtual address in the range 0x00_0000_0000 to 0x03_FFFF_FFFF directly maps to the physical address of the same range. The address range represents physical memory and can only be accessed in privileged mode.

Caches

Twin-64 is a cache-visible architecture that supports both unified and separate instruction and data caches. While the hardware directly manages cache line insertion and replacement, software has explicit control over flushing and purging cache line entries. Twin-64 caches are virtually indexed and physically tagged. To accelerate memory access, the cache uses the virtual address to index into the arrays in parallel with the TLB lookup. A cache hit occurs when the physical address tag in the cache matches the physical page address from the TLB.

The architecture does not mandate specific cache line sizes, cache organization, or hierarchy models. Instead, it defines instructions for cache operations, such as deletion or flushing of cache lines.

Each TLB entry indicates whether the corresponding data should be cached. For segments 0 to 15, where the TLB is bypassed, the hardware determines the caching behavior. Memory address ranges are always cached, whereas the I/O address range is not cached.

System Memory

Virtual Address Space

Twin-64 features a 52-bit virtual address range, organized into a 20-bit segment identifier and a 32-bit offset within that segment. The following figure illustrates the structure of a virtual address and how the segment is selected.

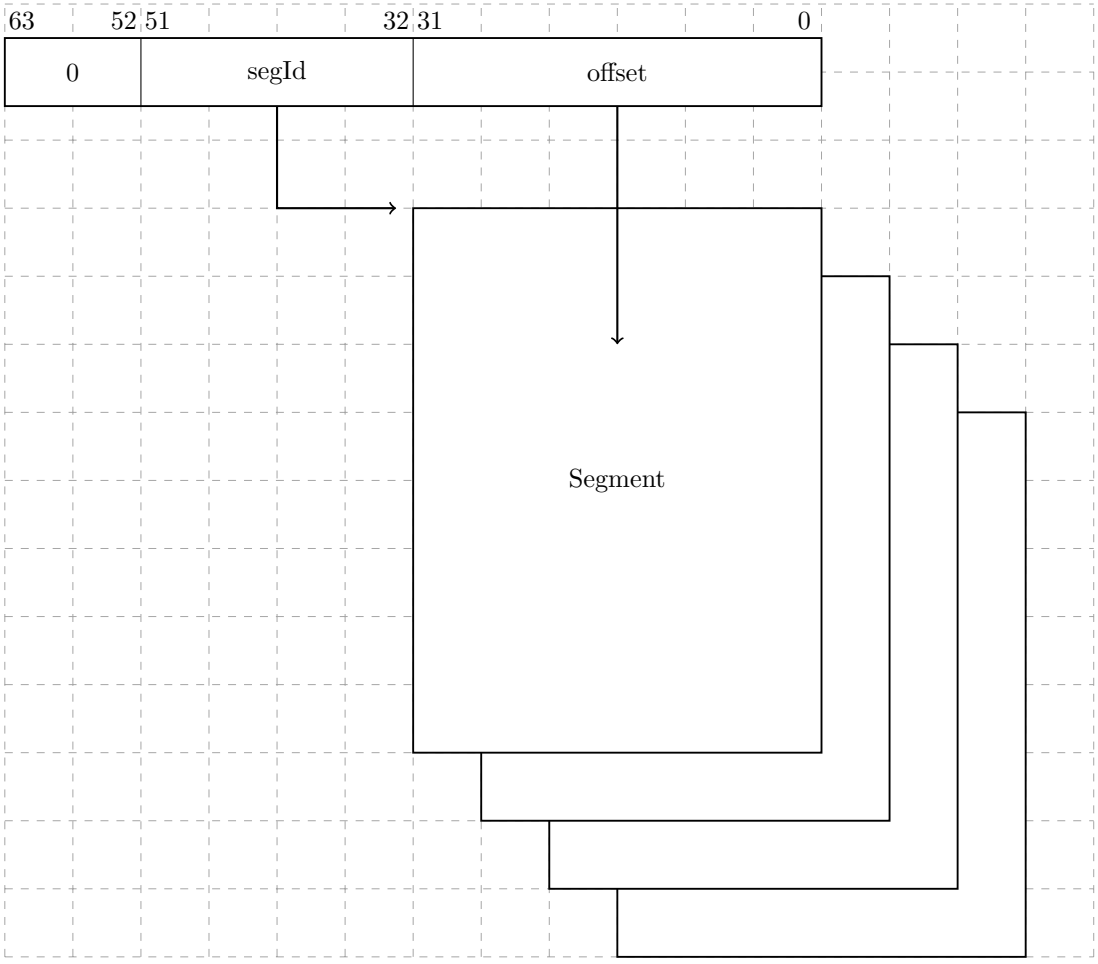


Figure 8: Virtual Memory Address Range

A virtual address is translated into a physical address. For translation purposes, the virtual address range can also be viewed as a set of virtual pages. The architected page size is 16 KB, with a 38-bit virtual page number and a 14-bit page offset.

Physical Address Space

Twin-64 architecture features a 36-bit physical memory address range which allows a maximum physical memory of up to 64 GBytes.

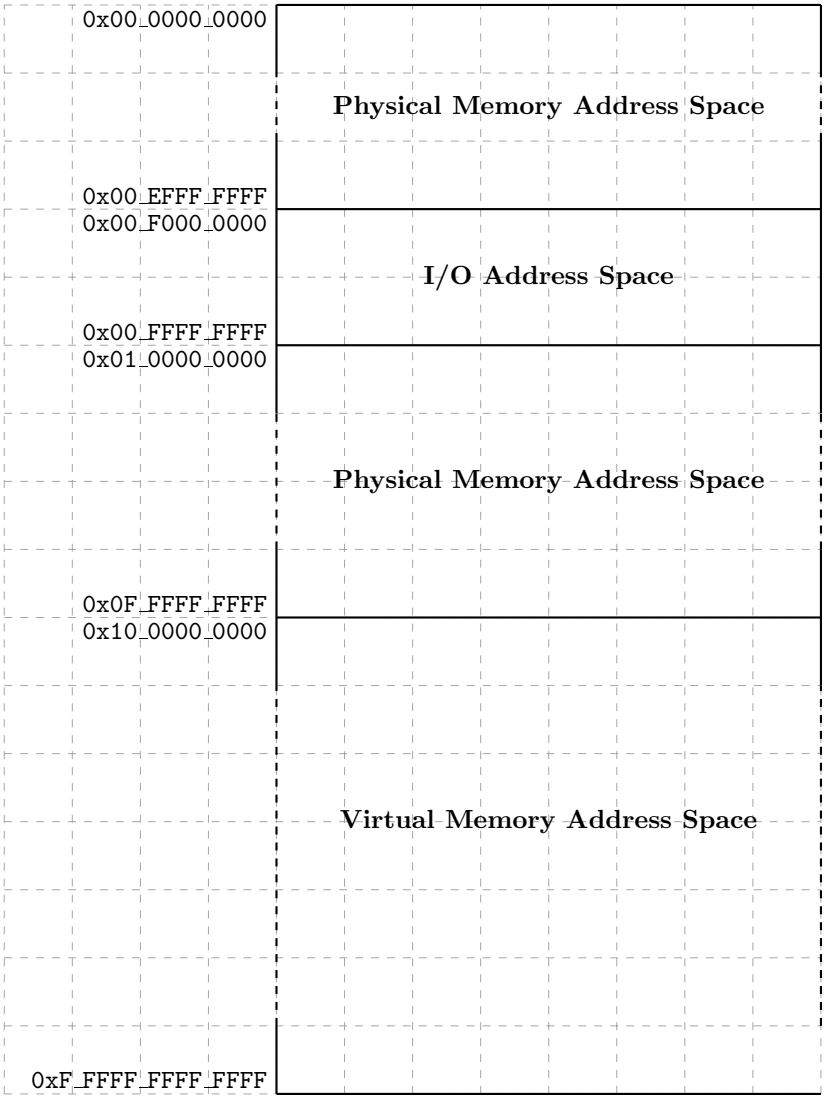


Figure 9: Physical Memory Address Range

Segment IDs 015 represent the physical address space, with a maximum size of 64 GB. These segments are directly mapped to physical addresses, without TLB translation or access-rights checking. Only privileged-mode code can access these segments. Control Register 0 contains a field that encodes the actual memory size installed in the system.

Twin-64 uses a memory-mapped I/O architecture. The I/O address space occupies the top 256 MB of the first segment. I/O modules allocate their registers and storage within this range, and access to this area is uncached.

The remaining address space is dedicated to virtual memory. Virtual storage is allocated dynamically, and the TLB provides the necessary translation to physical addresses while enforcing access rights.

Addressing and Access Control

The CPU generates 52-bit virtual addresses. Each virtual memory access must be translated and checked for valid access rights. This chapter describes the address translation process, as well as the access-rights and protection mechanisms.

Address Translation

Each virtual memory access generated by the processor must be translated into a physical address. The translation process maps a virtual page to a physical page. Virtual pages in the first 16 segments bypass the TLB, meaning that the virtual address is identical to the physical address.

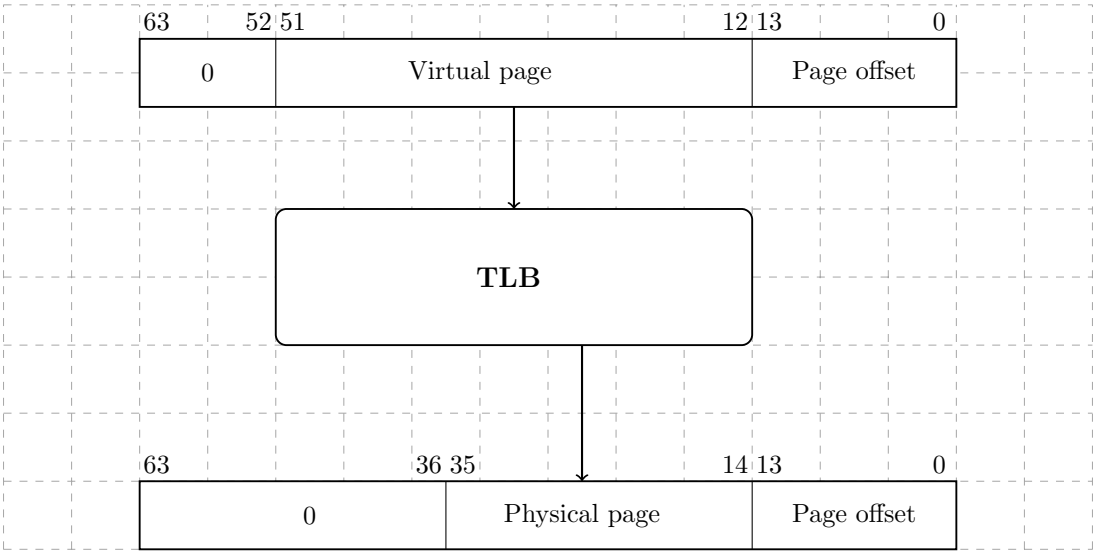


Figure 10: Address translation

Access Control

Twin-64 implements a protection model along two dimensions. The first dimension is **access control** and **privilege level**. Each page is associated with a page type: read-only, read-write, execute, or gateway. The page type is encoded in the AccType field, where 0 represents read-only access, 1 represents read-write access, 2 represents execute access, and 3 represents gateway access.

Each memory access is checked against the allowed access type defined in the page descriptor. There are two privilege levels: user mode and supervisor mode. The combination of access type and privilege level forms the access control information, which is verified for every instruction and data memory access.

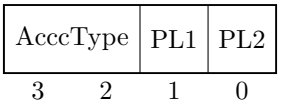


Figure 11: Access control information field

For read access, the privilege level must be at least as high as the PL1 field; for write access, it must be at least as high as PL2. Exceptions are execute and gateway pages, where write access is allowed only for privileged code. For execute access, the privilege level must be at least PL1 and no higher than PL2. If the instructions privilege level falls outside these bounds, a privilege violation trap is raised. If the page type does not match the instructions access type, an access violation trap is raised. In both cases, the instruction is aborted, and a memory protection trap occurs.

Table 4: Access type checking

Type	**read**	**write**	**execute**
`0 - read-only`	`PL <= PL1`	`not allowed`	`not allowed`
`1 - read-write`	`PL <= PL1`	`PL <= PL2`	`not allowed`
`2 - execute`	`PL <= PL1`	`PL == 0`	`PL2 <= PL <= PL1`
`3 - gateway`	`PL <= PL1`	`PL == 0`	`PL2 <= PL <= PL1`

Access Protection

The second dimension of protection is the **protection ID**. Twin-64 allows the processor to store up to eight protection ID values in the control registers `CR4` through `CR7`, which are accessible to the currently executing process. The protection ID is identical to a segment ID. For each access to a segment with the protection-check bit set, at least one of the protection ID registers must match the segment ID; otherwise, a protection violation trap is raised.

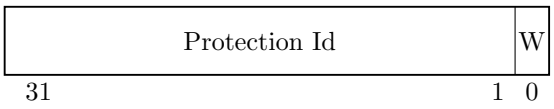


Figure 12: Protection Id

When protection ID checking is enabled, the eight protection identifiers are compared with the segment ID of the target address to validate an access. The rightmost bit of each protection ID register serves as a write-disable (W) bit. When the W bit is set, that protection ID cannot be used to grant write access. If the PSW P-bit is cleared, protection ID and write-disable checks are bypassed.

Protection ID checking is typically used to enforce controlled access to shared or private memory regions. A common example is the stack data segment, which is accessible at user privilege level. Since the CPU implements a global virtual memory address space, any task could potentially access the stack of another task. By enabling protection ID checking and loading the segment

ID into one of the protection ID registers for the stack owner process, only the corresponding user task is allowed to access its stack data segment with a matching protection ID.

Control Flow

In Twin-64, control flow normally passes to the next sequential instruction in memory unless altered by branch instructions or interruptions. To the programmer, the CPU appears to execute instructions in the order in which they are stored in memory. However, hardware implementations may internally execute instructions out of order. This chapter provides a logical view of the execution model. The sections on branching and interruptions describe how control flow can be altered during program execution.

Unconditional Branches

Unconditional branch instructions alter control flow, and may optionally save the return point to a general register. The branch target address can be computed in two modes:

- **Instruction-relative branches:** The target address is computed by adding an offset to the current instruction address.
- **Segment-relative branches:** The target address is computed relative to the start of the segment, which is address zero.

Both methods use 32-bit unsigned arithmetic, with wrap-around on overflow. For example, suppose the current instruction is at address `0xFFFF_FFFC` and the branch offset is `0x10`. The target address is:

$$\texttt{0xFFFF\_FFFC} + \texttt{0x10} = \texttt{0x1\_0000\_000C}$$

Because address arithmetic is performed modulo 2^{32}, the high-order carry is discarded. The effective target is therefore `0x0000_000C`, and execution continues at that address.

Conditional Branches

Conditional branch instructions combine a comparison or test operation with an Instruction-relative branch. If the condition is met, control transfers to the target instruction. Otherwise, execution continues with the next sequential instruction.

Linkage

Certain branch instructions provide a return path for procedure calls. The return point is the instruction following the branch instruction. The linkage mechanism applies to both intra-segment and inter-segment branches. The return point (segment and offset) is saved in the specified target general register.

Branching and Segments

A branch address consists of a segment identifier and an offset:

- **Intra-segment branches:** Unconditional branches within the same segment. Unconditional branches are always within the same segment.
- **Inter-segment branches:** Unconditional branches into another segment, performed by setting the target address segment and offset portion.

Privilege Level Changes

Branch instructions may change the privilege level. Instructions may only access data or execute code when the current privilege level matches the required level. Switching between levels is often implemented using a trap operation to higher-privileged software such as the operating system kernel. However, traps are expensive, as they typically flush the CPU pipeline when invoking a trap handler.

Twin-64 implements a gateway mechanism for privilege changes. An unconditional branch instruction with the `.G` gateway option raises the privilege level when executed on a gateway page. The branch target continues execution with the privilege level specified by the gateway page, and the privilege bit in the PSW is set. Returning from a higher privilege level to code in a lower-privilege page automatically lowers the current privilege level.

Each instruction fetch compares the privilege level of the code page with the PSW `P`-bit:

- If the page requires a higher privilege level than the PSW, a privilege violation trap occurs.
- If the page requires a lower privilege level than the PSW, the CPU automatically demotes execution to the lower level.

Traps Associated with Branches

Branch instructions may cause traps based on PSW bits. If the PSW `T`-bit is set and a branch is taken, a *taken-branch trap* occurs. This mechanism is primarily intended for debugging.

Interruptions

Twin-64 features a simple but powerful interrupt system. Interruptions in Twin-64 are divided into four classesfaults, traps, interrupts, and checksthat may occur during instruction execution.

- Fault - The current instruction cannot be carried out due to a system problem, such as the absence of a page from main memory. After the system problem has been corrected, the faulting instruction should execute as intended. Faults are synchronous with respect to the instruction stream.
- **Trap** - There are two kind of traps. The types are traps raised because the function requested by the current instruction cannot or should not be carried out. A good example is the arithmetic signed overflow trap. Such an instruction is normally not re-executed. The second type of traps are raised so that the system can intervene before the or after the instruction is executed. An example for this type of trap is the debugging breakpoint trap. Traps are synchronous with respect to the instruction stream.
- **Interrupt** - An external entity such as an I/O module requires attention. Interrupts are asynchronous with respect to the instruction stream.
- **Check** - The processor has detected an internal fault or malfunction. Checks can be either synchronous or asynchronous with respect to the instruction stream.

All four classes of interruptions are handled in the same way.

Interrupt Handling

Handling of an interrupt is implemented as a fast context switch. When an interruption occurs, the hardware saves the PSW at the time of the interrupt to the IPSW control register. PSW in effect at the time of the interruption is saved in the IPSW. Depending on the interrupt type, this can be the current instruction of the next instruction. All status bits, which are part of the PSW, are saved too. The hardware will then set the PSW status bits as follows. The M bit is set if the interruption is a high priority machine check. All other bits are set to zero. Depending on the trap type, additional data is saved to the interrupt argument control registers IARG0 and IARG1.

The control register `IVA` holds the virtual address of the interrupt vector table. Execution of the interrupt handler begins at the address computed using the trap number:

$$\texttt{IVA} + (32 * \texttt{trap number})$$

Each entry consists of 8 instructions that can be executed. It is the responsibility of interruption handlers to unmask external interrupts as soon as possible, so as to minimize interrupt latency. The interruptions are categorized into four groups.

Example. Suppose the interrupt vector table base address is:

$\texttt{IVA} = \texttt{0x0000\_1000}$

Now, if a **trap number 5** (Illegal Instruction) occurs, the handler entry point is calculated as:

$\texttt{0x0000\_1000} + (32 * 5) = \texttt{0x0000\_10A0}$

Execution of the interrupt handler starts at `0x0000_10A0`.

Interrupt Vector Table

Table 5: Trap Table

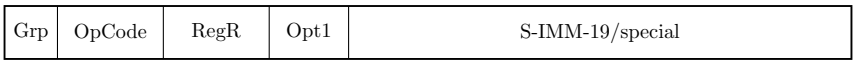
Trap	Name	Instruction Adr	Arg0	Arg1
1	Machine Check	current instruction	check type	-
2	Power Failure	current instruction	-	-
3	Recovery Counter	current instruction	-	-
4	External Interrupt	next instruction	-	-
5	Illegal instruction	current instruction	instruction	-
6	Privileged operation	current instruction	instruction	-
7	Privileged register	current instruction	instruction	-
8	Integer Overflow	current instruction	instruction	-
9	Instruction TLB miss	current instruction	-	-
10	Non-access Instruction TLB miss	current instruction	-	-
11	Instruction memory protection	current instruction	-	-
12	Data TLB miss	current instruction	instruction	target address
13	Non-access Data TLB miss	current instruction	instruction	target address
14	Data memory access rights	current instruction	instruction	target address
15	Data memory protection	current instruction	instruction	target address
16	Page reference	current instruction	instruction	target address
17	Alignment	current instruction	instruction	target address
18	Break Instruction	current instruction	instruction	-
19	Taken branch	next instruction	instruction	-

Instruction Set Overview

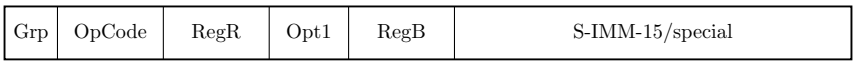
This chapter presents the Twin-64 instruction set. Following the typical RISC design, it features a fixed-length format with a small number of regular instruction types. Instructions are grouped into four categories: arithmetic and logical, load and store, branch, and special and system instructions.

Instruction Formats

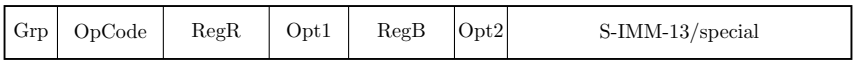
Twin-64 employs a small number of instruction formats, which are classified according to the number of register operands and the length of the immediate value field. The **signed immediate S19 / special** format provides one register operand and a 19-bit signed immediate field. This format is typically used for branch instructions.



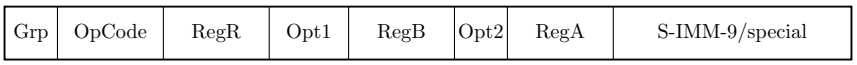
The **signed immediate S15 / special** instruction format provides two register operands and a 15-bit signed immediate field. This format is used both by conditional branch instructions and by instructions that operate on a register and an immediate value.



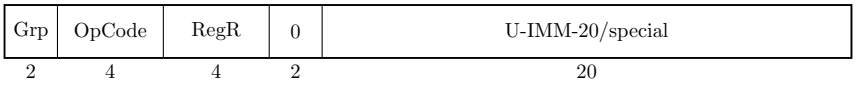
The **signed immediate S13 / special** instruction format provides two registers, an additional option field, and a 13-bit signed immediate field. Some instructions use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions, where the address is formed by a base register plus a 13-bit signed offset.



The **register/signed immediate S9 / special** instruction format provides three registers, an additional option field, and a 9-bit signed immediate field. Some instructions use the S-IMM-9 field for special encodings instead of a signed value. This instruction format is used for all computational instructions that require three registers.



The **unsigned immediate U20** format is used by the immediate instruction group to provide a 20-bit unsigned value. This value is then placed at specific positions in the target register `Reg R`.



Arithmetic Instructions

The arithmetic instructions operate on two signed 64-bit operands and store the result in the target register. There are two instruction layouts: signed immediate S15 and register-based. The **immediate operand instruction format** adds the sign-extended 15-bit value to `RegB`. The **register instruction format** uses two registers as input operands, performs the operation, and stores the result in a third register.

The `ADD` and `SUB` instructions perform signed 64-bit integer arithmetic, with numeric overflow triggering an overflow trap. There are no operations on unsigned quantities. The `SHLxA` and `SHRxA` instructions perform an arithmetic left or right shift on the input operand and add another operand to the shifted result, storing the final value in the target register. The `CMP` instruction compares two registers for equality, inequality, less than, and less than or equal conditions, storing the result in the target register.

Bit Operations Instructions

The bit operation instructions fall into two groups. The `AND`, `OR`, and `XOR` instructions perform the logical operations indicated by their names. Like the arithmetic instructions, they have both an immediate instruction format and a register instruction format.

The second group implements bit manipulation operations. The `EXTR` instruction extracts a bit field from a register and places it in another register; optionally, the extracted field can be sign-extended. The `DEP` instruction deposits a bit field into a target register. The `DSR` instruction concatenates two general-purpose registers, shifts the resulting 128-bit quantity to the right, and stores the lower 64 bits in the target register.

Immediate Instructions

The `LDI` and `ADDIL` instructions are used for forming large constants. With a 32-bit instruction word, it is not possible to encode even a full 32-bit constant in a single instruction. The `LDI` instruction includes a qualifier to load a constant value at a defined position in the target register. The `opt` field allows selection of the specific fields in the target register where the constant value is stored. Using a short sequence of instructions, any 64-bit value can be loaded.

The `ADDIL` instruction adds the upper portion of a 32-bit immediate value to the lower 32-bit half of a general register. This instruction is primarily used for address arithmetic, enabling access to any location within a segment. The `LDO` instruction loads an offset into a general register. The address is formed by adding the signed immediate value to a base register. Address arithmetic is performed as unsigned 32-bit arithmetic, wrapping around within the 32-bit range.

Memory Reference Instructions

Twin-64 is a registermemory architecture. It supports the common load/store instructions as well as instructions that combine an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register plus offset mode, and the second is the base register plus register index mode.

The **indexed instruction format** loads the content of the memory address formed by adding the scaled 13-bit immediate field to the base register `RegB`, and adds the fetched data value to `RegA`. The `dw` field specifies the data width: a `dw` value of 0 loads an 8-bit value, 1 loads a 16-bit value, 2 loads a 32-bit value, and 3 loads a 64-bit value. In all cases, the fetched value is sign-extended to 64 bits. Additionally, the `dw` field scales the offset by left-shifting the immediate field according to the data width. For example, a `dw` value of 3 loads a double word from an address formed by shifting the 13-bit offset by three bits before adding it to the base register.

The **register-indexed instruction format** loads the content of the memory address formed by adding `RegX` to the base register `RegB`. The `dw` field specifies the data width, as in the indexed instruction format. The offset in `RegX` is interpreted as a byte offset, independent of the `dw` value. In both instruction formats, the fetched data is sign-extended to 64 bits.

The `LD` and `ST` instructions are the standard load and store instructions. In addition to the two addressing modes described above, they also support base register modification. Depending on the sign of the offset, the base register is updated either before or after computing the effective address. See the instruction descriptions for more details.

The `LDR` and `STC` instructions provide the base support for a pipeline-friendly method for semaphore operations. They only support the indexed instruction mode with double-word access, and the base register modification option is not available.

The `ADD`, `SUB`, `AND`, `OR`, `XOR`, and `CMP` instructions perform the respective operations as described above, with one operand fetched from memory using the indexed addressing modes.

Control Flow Instructions

Control flow is implemented through a set of branch instructions, which can be classified into unconditional and conditional branches.

Unconditional branches fetch the next instruction address from the computed branch target. The `B` and `BR` instructions perform an instruction-relative branch. Both instructions can optionally store the return address in a general register.

The `BV` instruction performs a vectored branch. The content of a general register is added to a base register to form the branch target.

The `BB`, `CBR`, `MBR`, and `ABR` instructions implement conditional branching. If the specified condition is met, a branch to the target address is performed. The target address is always a local address, with the offset being instruction-address relative. Conditional branches adopt a static prediction model where forward branches are assumed not taken, while backward branches are assumed taken.

All branch address arithmetic is performed as 32-bit unsigned arithmetic with wraparound. Adding branch offsets will not overflow into the segment Id portion of the address.

System Instructions

The final instruction group is the **special/system** instruction group. The system control instructions manage CPU resources such as the TLB, cache, and other hardware components. Most instructions in this group require the processor to run in privileged mode.

The `MFCR` and `MTCR` instructions are used to transfer data to and from a control register. Access to some control registers requires privileged mode. The `MFIA` instruction accesses the current instruction address offset, which is important for position-independent code. The `RSM` and `SSM` instructions allow setting or clearing an individual accessible bit in the status portion of the processor state.

The `LDPA`, `PRBR`, and `PRBW` instructions are used to obtain information about the virtual memory system. The `LDPA` instruction returns the physical address of the virtual page, if present in memory. The `PRBx` instructions test whether the desired access mode to an address is allowed.

The translation lookaside buffers and caches are managed by the `ITLB`, `PTLB`, `FCA`, and `PCA` instructions. The `ITLB` and `PTLB` instructions insert or remove translations from the TLB. The `FCA` and `PCA` instructions manage the instruction and data caches, providing options to flush or purge a cache line. There is no instruction to insert data into a cache, as cache insertion is fully controlled by hardware.

The `RFI` instruction restores the processor state from the control registers. Optionally, general registers stored in reserved control registers will also be restored.

The `DIAG` instruction issues hardware-specific implementation commands. An example is the memory barrier command, which ensures that all memory access operations complete before subsequent instructions execute.

The `TRAP` instruction raises a software exception and is typically used to implement a debugging breakpoint.

Instruction Set Descriptions

This chapter describes the Twin-64 instruction set. Each instruction description includes the instruction word layout, a brief description of the instruction, and a high-level pseudo code representation of its operation. The pseudo code routines used in the instruction descriptions are listed in the appendix. Each instruction operation is described using C-style pseudo code. The pseudo code references registers by their class and index. For example, `GR[5]` denotes general register 5, and `CR[1]` denotes control register 1.

Some registers also have dedicated names. For example, the shift amount control register is labelled `SAR`. A subfield of an instruction is selected by adding a dot and the field name in square brackets. For instance, the interrupt enable bit in the PSW register is described as `PSW.[I]`.

Additional options for an instruction are specified by a dot followed by a sequence of characters in the assembler syntax. Each character represents a defined option for the instruction. For example, the `AND` instruction can negate its result; in assembler notation, this is written as `AND.N ...`. The order of the individual characters does not matter. All defined options are listed as a character set.

Reserved fields in an instruction word are reserved for future enhancements and should be filled with zero. The instruction explicitly shows the numeric value of these fields. Any other value in a reserved field will result in undefined behavior.

ABR - Add and branch

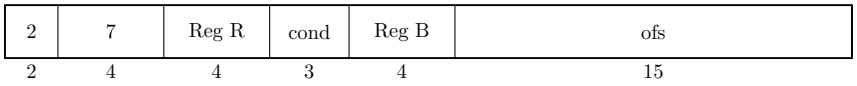
Syntax

`ABR.<cond> RegR, RegB, target`

where cond is EQ, LT, NE or LE.

Format

The ABR instruction uses the following format:



Description

The `ABR` instruction adds `RegB` to `RegR` and stores the result in `RegR`. If the condition specified in the `cond` field is satisfied, execution continues at the current instruction plus the offset; otherwise, execution proceeds with the next instruction.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	RegR == 0
1	LT	RegR < 0
2	NE	RegR != 0
3	LE	RegR <= 0

Operation

```
RegR <- RegR + RegB;
if ( cond( RegR )) IA <- Ia + ofs;
else               IA <- Ia + 4;
```

Exceptions

Overflow trap.

Notes

None.

ADD Integer Addition

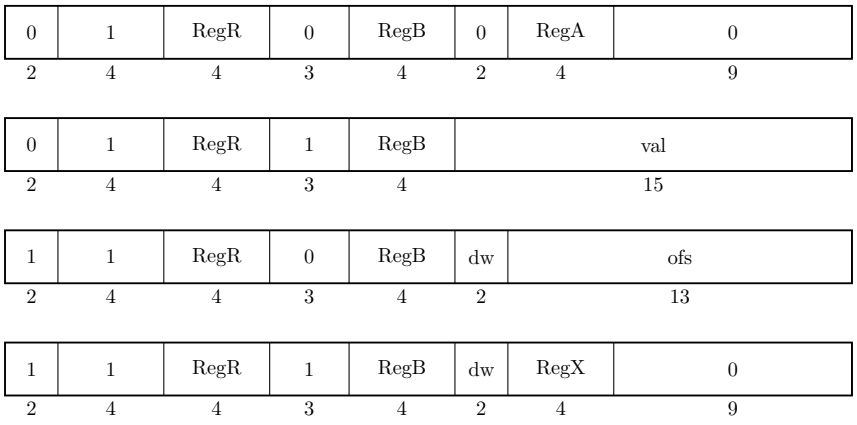
Syntax

```
ADD RegR, RegB, RegA
ADD RegR, RegB, val

ADD[.D/W/H/B] RegR, ofs ( RegB )
ADD[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The ADD instruction uses the register, the immediate-15, the indexed and the register indexed instruction formats.



Description

The `ADD` instruction adds two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-referencerelated trap.

Operation

```
RegR <- RegB + RegA;          ( register format )
RegR <- RegB + immOperand( ); (immediate format)
RegR <- RegR + memOperand( ); (indexed formats)
```

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

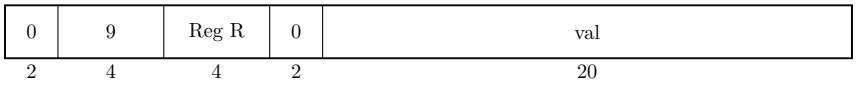
ADDIL - Add immediate left

Syntax

```
ADDIL RegR, val
```

Format

The ADDIL instruction uses the immediate-20 instruction format.



Description

The `ADDIL` instruction adds an immediate value to a general register using 32-bit unsigned arithmetic. The 20-bit immediate value is shifted left by 12 bits (padded with zeros on the right) before being added to `RegR`. The resulting value is stored in the general register `GR1`. Any overflow that occurs during the addition is ignored.

Operation

```
RegR <- RegR + ( val << 12 );
```

Exceptions

None.

Notes

None.

AND - Boolean Operation

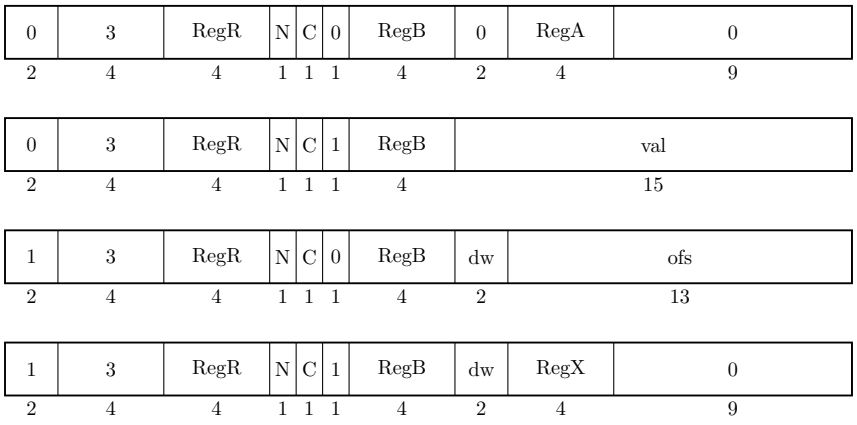
Syntax

```
AND[.C/N] RegR, RegB, RegA
AND[.C/N] RegR, RegB, val

AND[.C/N/D/W/H/B] RegR, ofs( RegB )
AND[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The AND instruction uses the register, the immediate-19, the indexed and the register indexed instruction formats.



Description

The `AND` instruction performs a bitwise AND operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegB`, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The Boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-referencerelated trap.

Operation

```
if ( instr.[C] ) RegB <- not( RegB );
RegR <- RegB & RegA;           (register format)
RegR <- RegB & immOperand();   (immediate format)
RegR <- RegR & memOperand();   (indexed formats)
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

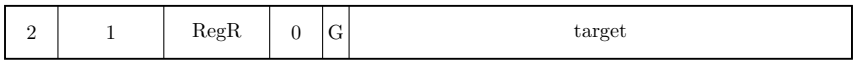
B Branch Unconditional

Syntax

```
B[.G] target [, RegR ]
```

Format

The `B` instruction uses the immediate-19 format.



Description

The `B` instruction performs an instruction-addressrelative branch. The target address is computed by sign-extending the immediate offset, shifting it left by two bits, and adding the result to the current instruction address.

If `RegR` is specified, it is loaded with the address of the following instruction before the branch occurs. If no register is specified, this field must be set to zero.

The optional `.G` form enables the *gateway branch* option. In this case, the processors privilege level may be raised to the privilege level stored in the TLB entry of the page from which the gateway instruction is fetched. The change is only performed if it results in a higher privilege level; otherwise, the current privilege level remains unchanged. If code translation is disabled, the privilege level is set to zero. The gateway instructions privilege level is also deposited into the general register `RegR`. Execution then continues at the computed target address with the (possibly new) privilege level.

Operation

```
??? revise

if ( RegR != 0 )
    RegR <- addOfs32( PSW.[IA], 4 );

if ( instr.[G] ) {
    new_priv <- TLB[PSW.[IA]].priv;
    if ( !code_translation_enabled )
        new_priv <- 0;
    if ( new_priv > PSW.[PL] )
        PSW.[PL] <- new_priv;
    RegR <- new_priv;   // deposit privilege level into RegR
}

PSW.[IA] <- addOfs32( PSW.[IA], target );
```

Exceptions

Instruction Address Trap
Instruction Privilege Violation Trap
Instruction TLB Miss Trap

Notes

None.

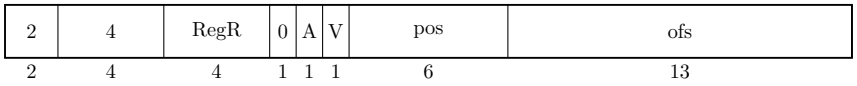
BB Branch on Bit

Syntax

```
BB[.T/F] ofs, pos
BB[.T/F] ofs, SAR
```

Format

The BB instruction uses the immediate-9 format.



Description

The BB instruction tests a bit in register RegR. The bit position can be specified directly by the pos field, or indirectly through the Shift Amount Register (SAR) if the A option is set.

If the V bit is set, the test succeeds when the selected bit is 1. If the V bit is clear, the test succeeds when the selected bit is 0.

When the condition is satisfied, execution branches to the target address, computed by sign-extending the 9-bit offset, shifting it left by two bits, and adding it to the current instruction address. If the condition is not satisfied, execution continues with the next sequential instruction.

Operation

```
if ( instr.[A] )
    pos <- SAR;
else
    pos <- instr.[pos];

bit_val <- testBit( RegR, pos );

cond <- ( instr.[V] ? (bit_val == 1) : (bit_val == 0) );

if ( cond )
    PSW.[IA] <- addOfs32( PSW.[IA], ofs );
else
    PSW.[IA] <- addOfs32( PSW.[IA], 4 );
```

Exceptions

Instruction Address Trap
Instruction TLB Miss Trap

Notes

None.

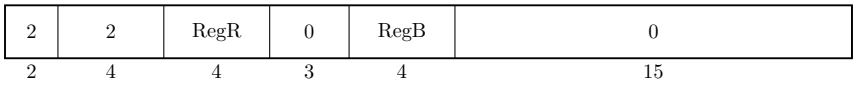
BR Branch Register

Syntax

```
BR RegB [, RegR ]
```

Format

The `BR` instruction uses the immediate-15 instruction format.



Description

The `BR` instruction performs an unconditional branch to a target address computed relative to the current instruction address (`IA`). The target address is formed by taking the contents of `RegB`, shifting it left by two bits, and adding it to `IA`.

If `RegR` is specified, it receives the return address (`IA + 4`) before the branch is taken.

Operation

```
RegR <- addOfs32( PSW.[IA], 4 );
PSW.[IA] <- addOfs32( PSW.[IA], RegB << 2 );
```

Exceptions

None.

Notes

None.

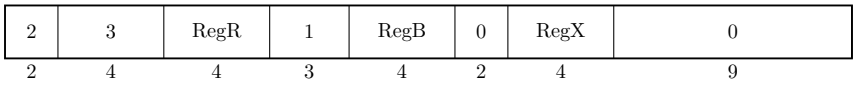
BV Branch Vectored

Syntax

`BV RegB [, RegR, RegX ]`

Format

The `BV` instruction uses the register + register indexed instruction format.



Description

The `BV` instruction performs an unconditional branch to the address stored in `RegB`.

The target is interpreted as an instruction address within the current code segment. If code translation is disabled, the target address is the absolute physical address.

Before branching, the next instruction address (`IA + 4`) is stored in `RegR`.

Because `BV` is a segment base relative branch, branching to a page with a different privilege level is possible. Branching from a lower privilege level to a higher one triggers an instruction protection trap. Branching from a higher privilege to a lower privilege level updates the privilege level in the processor status register. If no change is required, the privilege level remains unchanged.

Indexed formats may cause a memory-referencerelated trap. (???)

Operation

```
RegR <- addOfs32( PSW.[IA], 4 );
PSW.[IA] <- computeTarget( RegB, RegX );
```

Exceptions

Instruction Protection Trap
Memory Reference Trap (if indexed) (???)

Notes

None.

CBR - Compare and Branch

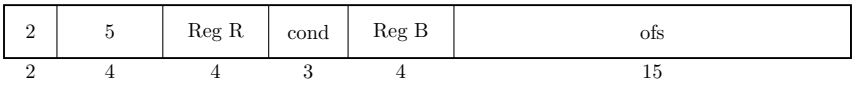
Syntax

`CBR.<cond> RegR, RegB, target`

where cond is EQ, LT, NE or LE.

Format

The CBR instruction uses the following format:



Description

The `CBR` instruction compares the contents of `RegB` with `RegR`. If the condition specified in the `cond` field is satisfied, execution continues at the current instruction plus the offset. Otherwise, execution proceeds with the next instruction. The registers themselves are not modified.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == RegB`
1	LT	`RegR < RegB`
2	NE	`RegR != RegB`
3	LE	`RegR <= RegB`

Operation

```
if ( cond( RegR, RegB )) IA <- IA + ofs;
else                     IA <- IA + 4;
```

Exceptions

None.

Notes

None.

CMP - Compare

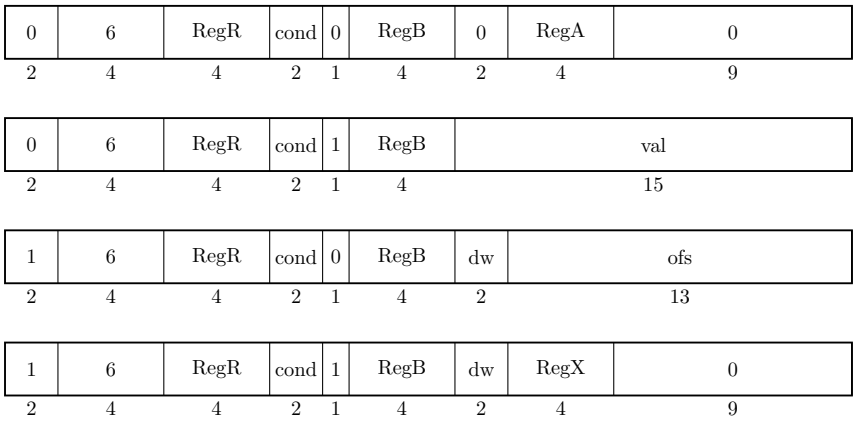
Syntax

```
CMP RegR, RegB, RegA
CMP RegR, RegB, val

CMP[.D/W/H/B] RegR, ofs ( RegB )
CMP[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The CMP instruction uses the register, the immediate-15, the indexed and the register indexed instruction formats.



Description

The `CMP` instruction compares two signed 64-bit operands. In register mode, it compares `RegB` and `RegA`; in immediate mode, it compares `RegB` and the immediate value; and in memory-reference formats, it compares `RegR` with the memory operand. The result of the comparison i stored in RegR. The instruction does not cause an arithmetic overflow but indexed instruction formats may cause a memory-referencerelated trap.

Conditions

...

Operation

```
RegR <- compare( RegB, RegA );          (register format)
RegR <- compare( RegB, immOperand());   (immediate format)
RegR <- compare( RegR - memOperand());  (indexed formats)
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

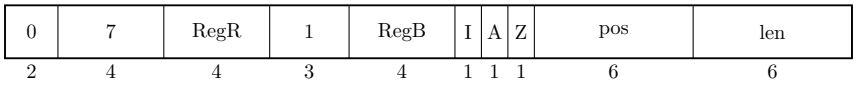
DEP - Deposit Bit Field

Syntax

```
DEP[.Z/I] RegR, RegB, pos, len
DEP[.Z/I] RegR, RegB, SAR, len
```

Format

The `DEP` instruction uses the special-13 instruction format.



Description

The `DEP` instruction deposits a bit field from general register `RegB` into `RegR` at the specified position.

The rightmost bit of the target field is specified by the `pos` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set. The length of the field is specified by the `len` instruction field.

If the `Z` bit is set, the target register `RegR` is cleared before depositing the field. If the `I` bit is set, the value to deposit is taken from the RegB field rather than a register in `RegB`.

Operation

```
if ( instr.[Z] )
    RegR <- 0;

if ( instr.[A] )
    pos <- shamt;
else
    pos <- instr.[pos];

if ( instr.[I] )
    RegR <- depositImm( RegR, RegB, pos, instr.[6:0] );
else
    RegR <- depositField( RegR, RegB, pos, instr.[6:0] );
```

Exceptions

None

Notes

None

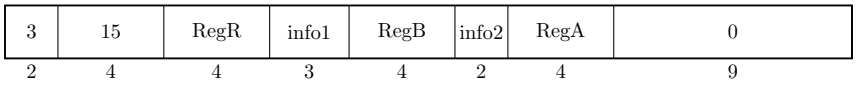
DIAG - Diagnose Instruction

Syntax

```
DIAG info1, RegB, info2, RegA
```

Format

The `DIAG` instruction uses the indexed instruction format, identical to the TRAP instruction.



Description

The `DIAG` instruction provides a mechanism for diagnostic and privileged operations.

The `info1` and `info2` fields specify the diagnostic function. The `RegB` and `RegA` fields may provide operands or data required by the diagnostic function. Any result is returned in `RegR`.

Operation

```
... // Implementation depends on diagnostic function code and
    operands
```

Exceptions

Depends on diagnostic function, operands, and addressing mode; may include memory reference or privilege level traps.

Notes

None

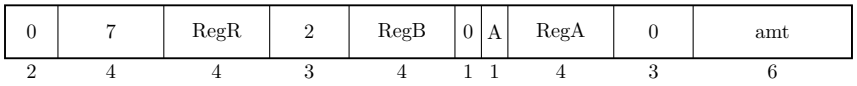
DSR Double Shift Right

Syntax

```
DSR RegR, RegB, RegA, amt
DSR RegR, RegB, RegA, SAR
```

Format

The `DSR` instruction uses the special-9 instruction format.



Description

The `DSR` instruction concatenates the contents of registers `RegB` (left) and `RegA` (right) and performs a right shift operation. The lower 64 bits of the shifted result are stored in `RegR`.

The shift amount is specified by the `amt` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set.

Operation

```
if ( instr.[A] )
    tmp <- shamtReg;
else
    tmp <- instr.[amt];

RegR <- doubleShiftR( RegB, RegA, tmp );
```

Exceptions

None

Notes

None

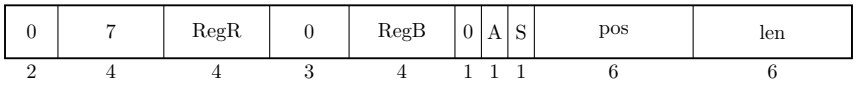
EXTR - Extract Bit Field

Syntax

```
EXTR[.S] RegR, RegB, pos, len
EXTR[.S] RegR, RegB, SAR, len
```

Format

The `EXTR` instruction uses the special-9 instruction format.



0	7	RegR	0	RegB	0	A	S	pos	len
2	4	4	3	4	1	1	1	6	6

Description

The `EXTR` instruction extracts a bit field from general register `RegB`.

The rightmost bit of the field is specified by the `pos` instruction field. If the `A` bit is set, the position value is taken from the Shift Amount Register (`SAR`) instead of the instruction field.

The length of the field is specified by the `len` instruction field. The extracted bit field is stored right-justified in `RegR`. If the `S` bit is set, the extracted value is sign-extended.

Operation

```
if ( instr.[A] )
    tmp <- SAR;
else
    tmp <- instr.[pos];

RegR <- extractField( RegB, tmp, len );

if ( instr.[S] )
    RegR <- signExtend( RegR, len );
```

Exceptions

None

Notes

None

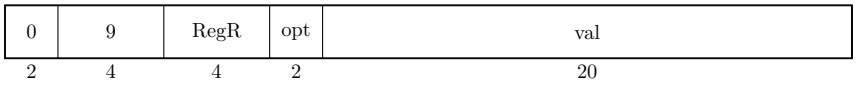
LDIL - Load Immediate left

Syntax

`LDIL[.L/S/U] RegR, val`

Format

The `LDIL` instruction uses the immediate-20 instruction format.



Description

The `LDIL` instruction loads an immediate value into the general register `RegR`. The optional suffix specifies how the immediate is positioned in the register:

??? unclear text ...

- `.L` Load into lower 20 bits and shift left by 12.
- `.S` Load into middle 20 bits and shift left by 32.
- `.U` Load into upper 12 bits and shift left by 52.

Operation

```
RegR <- ((val & 0xFFFFF) << 12)   // .L option
RegR <- ((val & 0xFFFFF) << 32)   // .S option
RegR <- ((val & 0xFFF)   << 52)   // .U option
```

Exceptions

None

Notes

None

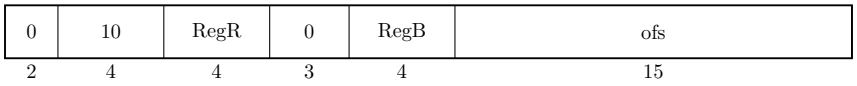
LDO Load Offset

Syntax

`LDO RegR, ofs(RegB)`

Format

The `LDO` instruction uses the immediate-15 instruction format.



Description

The `LDO` instruction loads a value from memory using an offset addressing mode. The effective address is calculated by adding the signed immediate offset `ofs` to the contents of `RegB`. The loaded value is then stored in `RegR`.

Operation

```
RegR <- addOfs32(RegB, ofs);
```

Exceptions

None.

Notes

None

MBR - Move and Branch

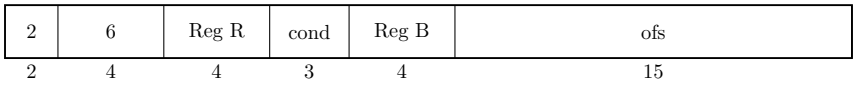
Syntax

`MBR.<cond> RegR, RegB, target`

where cond is EQ, LT, NE or LE.

Format

The MBR instruction uses the following format:



Description

The `MBR` instruction copies the contents of `RegB` into `RegR`. If the condition specified in the `cond` field evaluates true for the new value in `RegR`, execution continues at the current instruction plus the offset. Otherwise, execution resumes with the next instruction.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == 0`
1	LT	`RegR < 0`
2	NE	`RegR != 0`
3	LE	`RegR <= 0`

Operation

```
RegR <- RegB;
if ( cond( RegR )) IA <- IA + ofs;
else               IA <- IA + 4;
```

Exceptions

None.

Notes

None.

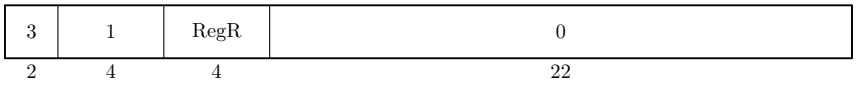
MFIA - Move from Instruction Address

Syntax

```
MFIA RegR
```

Format

The `MTCR` instruction uses the control-register instruction format.



Description

The `MFIA` instruction copies the current instruction address to register `RegR`.

Operation

```
RegR <- PSW.[IA];
```

Exceptions

None.

Notes

None.

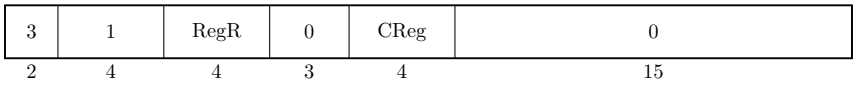
MFCR Move From Control Register

Syntax

```
MFCR RegR, CRegB
```

Format

The `MFCR` instruction uses the control-register instruction format.



Description

The `MFCR` instruction copies the contents of control register `CRegB` to general register `RegR`. Some control registers are privileged; attempting to read them in user mode may cause a privilege exception.

Operation

```
RegR <- CRegN;
```

Exceptions

Privilege exception if a privileged control register is accessed in user mode.

Notes

None

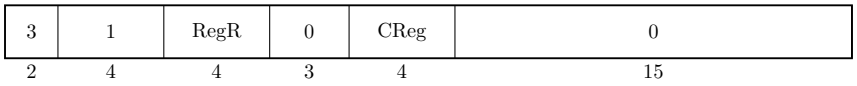
MTCR Move To Control Register

Syntax

```
MTCR CRegB, RegR
```

Format

The `MTCR` instruction uses the control-register instruction format.



Description

The `MTCR` instruction copies the contents of general register `RegR` to control register `CRegB`. Some control registers are privileged; attempting to write them in user mode may cause a privilege exception.

Operation

```
CRegN <- RegR;
```

Exceptions

Privilege exception if a privileged control register is written in user mode.

Notes

None

OR Boolean Operation

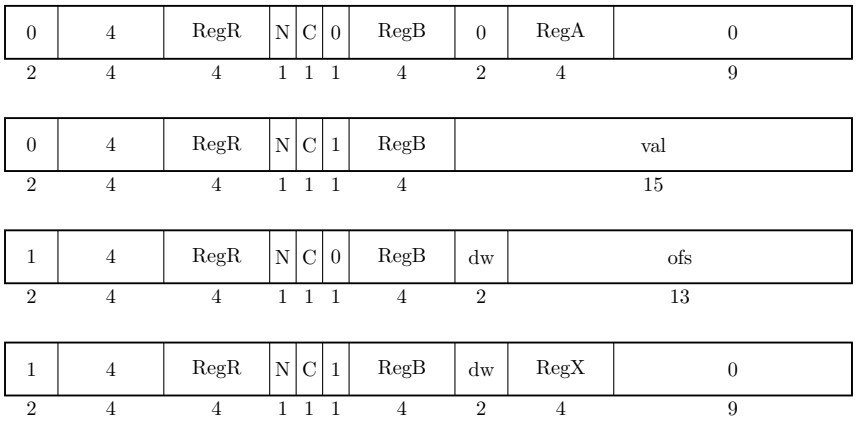
Syntax

```
OR[.C/N] RegR, RegB, RegA
OR[.C/N] RegR, RegB, val

OR[.C/N/D/W/H/B] RegR, ofs( RegB )
OR[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The OR instruction uses the register, the immediate-19, the indexed and the register indexed instruction formats.



Description

The `OR` instruction performs a bitwise OR operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegB`, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The Boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-referencerelated trap.

Operation

```
if ( instr.[C] ) RegB <- not( RegB );
RegR <- RegB | RegA;           (register format)
RegR <- RegB | immOperand();   (immediate format)
RegR <- RegR | memOperand();   (indexed formats)
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

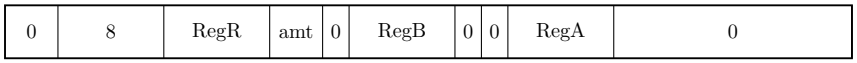
SHLxA Shift Left and Add

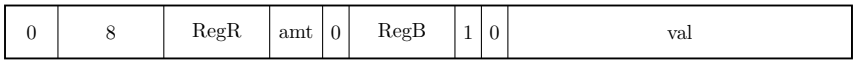
Syntax

```
SHLxA RegR, RegB, RegA
SHLxA RegR, RegB, SignedImmed13
```

Format

The `SHLxA` instruction supports both register and immediate formats.





Description

The `SHLxA` instruction shifts the contents of `RegB` left by the amount specified in the `amt` field.

In register format, `RegA` is added to the shifted value. In immediate format, if the I bit is set, the sign-extended `val` field is added instead.

The resulting value is stored in `RegR`.

Operation

```
RegR <- (RegB << amt) + RegA;                    (Register format)
RegR <- (RegB << amt) + immOperand();            (Immediate format
    )
```

Exceptions

Integer Overflow Trap

Notes

None

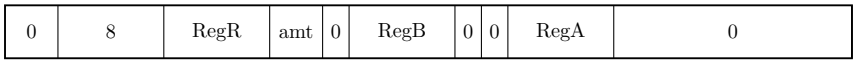
SHRxA - Shift Right and Add

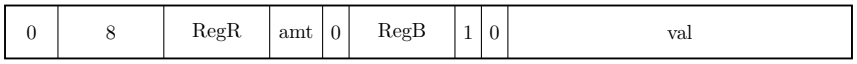
Syntax

```
SHRxA RegR, RegB, RegA
SHRxA RegR, RegB, val
```

Format

The `SHRxA` instruction supports both register and immediate formats.





Description

The `SHRxA` instruction shifts the contents of `RegB` right by the amount specified in the `amt` field.

In register format, `RegA` is added to the shifted value. In immediate format, if the I bit is set, the sign-extended `val` field is added instead.

The resulting value is stored in `RegR`.

Operation

```
RegR <- (RegB >> amt) + RegA;                (Register format)
RegR <- (RegB >> amt) + immOperand();        (Immediate format)
```

Exceptions

Integer Overflow Trap

Notes

None

SUB Integer Subtraction

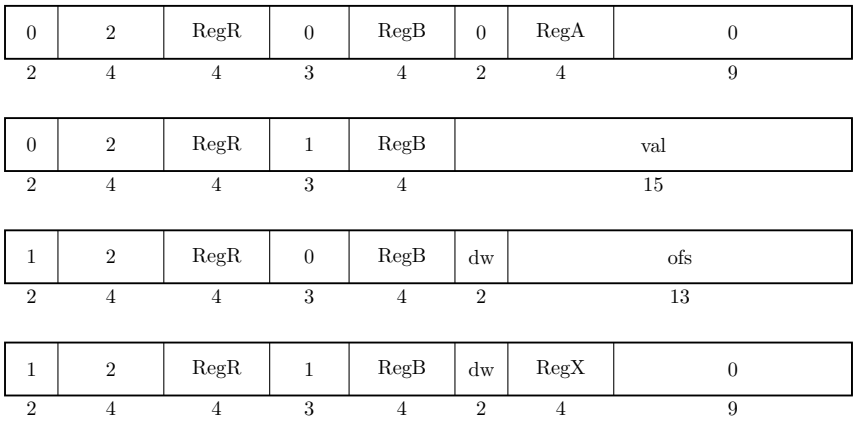
Syntax

```
SUB RegR, RegB, RegA
SUB RegR, RegB, val

SUB[.D/W/H/B] RegR, ofs ( RegB )
SUB[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The SUB instruction uses the register, the immediate-15, the indexed and the register indexed instruction formats.



Description

The `SUB` instruction subtracts two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-referencerelated trap.

Operation

```
RegR <- RegB - RegA;          ( register format )
RegR <- RegB - immOperand( ); (immediate format)
RegR <- RegR - memOperand( ); (indexed formats)
```

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

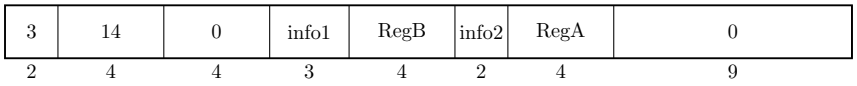
TRAP Trap Instruction

Syntax

`TRAP ...`

Format

The `TRAP` instruction uses the standard indexed instruction format.



Description

The `TRAP` instruction generates a synchronous trap to handle exceptional conditions. Indexed formats may trigger a memory reference type trap depending on the operands and addressing mode. The instruction may be used to invoke operating system routines or exception handlers.

Operation

```
... // Implementation depends on the trap vector and type
```

Exceptions

Depends on trap type and addressing mode; may include memory reference trap.

Notes

None

XOR Boolean Operation

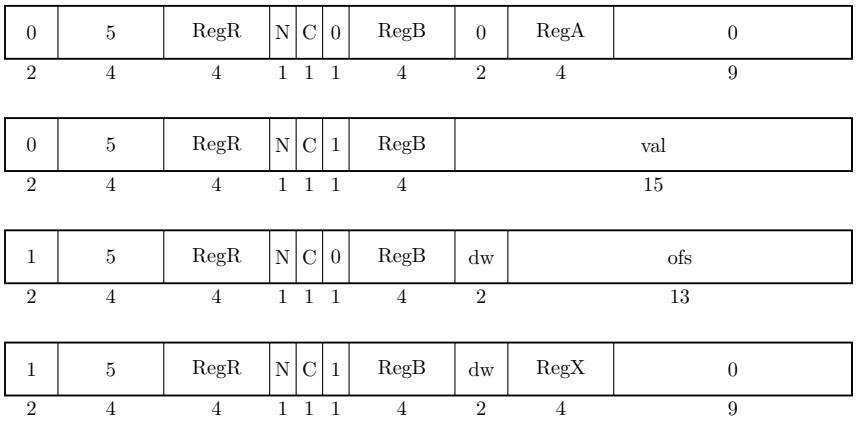
Syntax

```
XOR[.C/N] RegR, RegB, RegA
XOR[.C/N] RegR, RegB, val

XOR[.C/N/D/W/H/B] RegR, ofs( RegB )
XOR[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The XOR instruction uses the register, the immediate-19, the indexed and the register indexed instruction formats.



Description

The `XOR` instruction performs a bitwise XOR operation on two 64-bit operands and stores the result in the target register `RegR`. The result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The Boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-referencerelated trap.

Operation

```
if ( instr.[C] ) RegB <- not( RegB );
RegR <- RegB ^ RegA;           (register format)
RegR <- RegB ^ immOperand();   (immediate format)
RegR <- RegR ^ memOperand();   (indexed formats)
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

Input/Output

Concepts

Twin-64 features memory a mapped I/O architecture.

memory mapped

dedicated address space in physical memory

bus concept

module concept

modules have registers - load / store, however registers do not have to be 64 bit wide.

the system bus features up to 64 modules. that is: 64 HPA pages, need: 1Mb address range.

a module listens to three address ranges

- hard physical address range. a page. privileged.

- soft physical address range. allocated by the module for further space. aligned on a page boundary.

- broadcast address range. Mirrors the system bus range. A write to a register in that range applies to all corresponding registers of a module on the same bus.

modules are processors, memory and I/O cards.

we could keep processor stats in the HPA space

memory module has entire memory as SPA space

I/O Address Space

As shown in the overall physical memory layout, the physical memory address range dedicated to I/O modules is located from `0x00_F000_0000` to `0x00_FFFF_FFFF`. The following figure depicts the I/O memory address space ranges in more detail.s

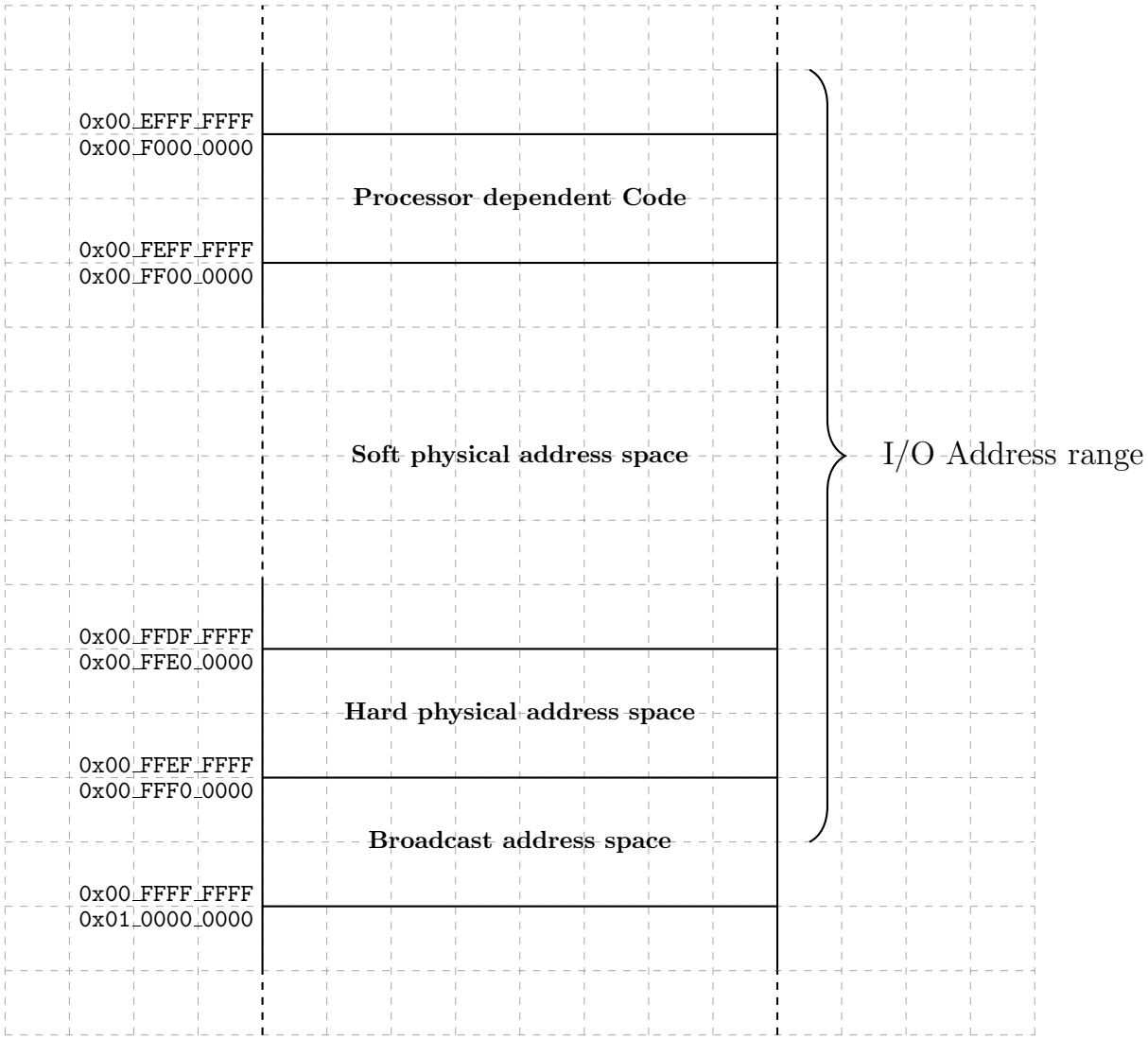


Figure 13: I/O Memory Address Range

The I/O address range is a 256Mbyte area in the first 4 Gbyte segment, i.e.segment 0. The area is divided into several ranges. The first range contains the processor dependent code. There is a set of library functions to manage the processor itself but also provide low-level primitives for the boot process and the operating system.

Hard physical address range

The **hard physical address** range contains a I/O page for each I/O modules installed. There room for 64 I/O modules.

Broadcast address range

The **broadcast address** range mirrors the hard physical address range. However, a write to these locations is a write operation to which all I/O modules will react.

Soft physical address range

The **soft physical address**range is an area where space is allocated to an I/O module. The I/O module reacts to a memory access operation in its configured soft physical address range.

I/O Elements

I/O elements are 128 bytes, i.e. 16 regs

I/O elements can also be two pages, one priv and one user level, mapped...

Appendix - Instruction Commentary

xxx

Appendix - Instruction Notation Routines

describe the routines used in the instruction set operation part...

Table 6: Instruction Notation Routines

Function	**Description**
...	...
...	...

Appendix -TLB and Caches

xxx